

ВСТУП ДО ASP.NET CORE

Мета: ознайомитися з основними принципами роботи .NET, навчитися налаштовувати середовище розробки та встановлювати необхідні компоненти, набути навичок створення рішень та проектів різних типів, набути навичок обробки запитів з використанням middleware.

Хід роботи:

Завдання 2. Створення проектів

Частина 1.

1. Створіть проект для консольного додатку з назвою ConsoleToWeb з використанням [dotnet CLI](#)
2. Перетворіть створений консольний додаток на веб-додаток
3. Опишіть виконані кроки у звіті

```
● PS C:\IPZ-23-4\ASP\lab_1> dotnet new console -n ConsoleToWeb
The template "Console App" was created successfully.

Processing post-creation actions...
Restoring C:\IPZ-23-4\ASP\lab_1\ConsoleToWeb\ConsoleToWeb.csproj:
Restore succeeded.
```

Рис. 1. Створення консольного додатку за допомогою терміналу та dotnet CLI

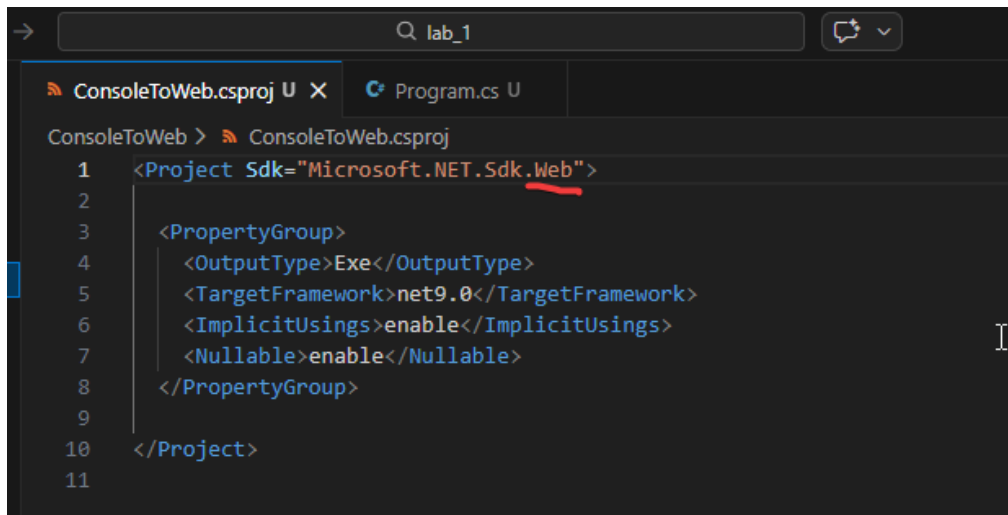


Рис. 2. Було змінено консольний додаток на веб

					ДУ «Житомирська політехніка».26.121.08.000 – Лр1						
Змн.	Арк.	№ докум.	Підпис	Дата							
Розроб.		Грушевицький Д.І.			Звіт з лабораторної роботи			Лім.	Арк.	Аркушів	
Перевір.		Українець М.О.							1	6	
Керівник								ФІКТ Гр. ІПЗ-23-4			
Н. контр.											
Зав. каф.											

Частина 2.

1. Створіть ASP.NET WebAPI проект без авторизації з назвою WebFromCli з використанням [dotnet CLI](#)
2. Реалізуйте GET-ендпоінт "/who", який повертатиме у відповідь ваше ім'я та прізвище
3. Реалізуйте GET-ендпоінт "/time", який повертатиме у відповідь поточний час на сервері
4. Наведіть лістинг реалізованих обробників у звіті

```
PS C:\IPZ-23-4\ASP\lab_1> dotnet new webapi -n WebFromCli
The template "ASP.NET Core Web API" was created successfully.

Processing post-creation actions...
Restoring C:\IPZ-23-4\ASP\lab_1\WebFromCli\WebFromCli.csproj:
Restore succeeded.

PS C:\IPZ-23-4\ASP\lab_1> cd .\WebFromCli\
```

Рис. 3. Створення WebAPI проекту з використанням dotnet CLI

```
WebFromCli > Program.cs
1 var builder = WebApplication.CreateBuilder(args);
2
3 var app = builder.Build();
4
5 app.MapGet("/who", () => "Денис Грушевицький");
6
7 app.MapGet("/time", () => DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss"));
8
9 app.Run();
```

Рис. 4. Реалізовані два GET-ендпоінти

```
localhost:5083/who
Денис Грушевицький
```

Рис. 5. Вивід GET-ендпоінта /who

```
localhost:5083/time
2026-02-27 00:09:03
```

Рис. 6. Вивід GET-ендпоінта /time

		Грушевицький Д.І.			ДУ «Житомирська політехніка».26.121.08.000 – Лр2	Арк.
		Українець М.О.				2
Змн.	Арк.	№ докум.	Підпис	Дата		

Частина 3.

1. Створіть ASP.NET MVC проект будь-яким зручним способом.
2. Реалізуйте контролер з назвою LabController
3. В створеному контролері реалізуйте обробник /info, який повертатиме View з даними про номер лабораторної роботи, тему, мету та ім'я та прізвище виконавця в табличному вигляді
4. Дані для відображення передати з контролера

```
PS C:\IPZ-23-4\ASP\lab_1> dotnet new mvc -n MvcLab
The template "ASP.NET Core Web App (Model-View-Controller)" was created successfully.
This template contains technologies from parties other than Microsoft, see https://aka.ms/aspnetcore/9.0-third-party-notices for details.

Processing post-creation actions...
Restoring C:\IPZ-23-4\ASP\lab_1\MvcLab\MvcLab.csproj:
Restore succeeded.
```

Рис. 7. Створення MVC проекту через консоль

```
MvcLab > Views > Lab > Info.cshtml
1  @{
2      ViewData["Title"] = "Інформація про лабораторну";
3  }
4
5  <div class="text-center">
6      <h2>Інформація про лабораторну роботу</h2>
7  </div>
8
9  <table class="table table-bordered table-striped mt-4">
10     <thead class="thead-dark">
11         <tr>
12             <th>Властивість</th>
13             <th>Значення</th>
14         </tr>
15     </thead>
16     <tbody>
17         <tr>
18             <td><strong>Номер лабораторної роботи</strong></td>
19             <td>@ViewBag.LabNumber</td>
20         </tr>
21         <tr>
22             <td><strong>Тема</strong></td>
23             <td>@ViewBag.Topic</td>
24         </tr>
25         <tr>
26             <td><strong>Мета</strong></td>
27             <td>@ViewBag.Purpose</td>
28         </tr>
29         <tr>
30             <td><strong>Виконавець</strong></td>
31             <td>@ViewBag.StudentName</td>
32         </tr>
33     </tbody>
34 </table>
```

Рис. 8. Створення View який виводить інформацію

```

MvcLab > Controllers > LabController.cs > {} MvcLab.Controllers > LabController > Info() : IActionResult
1  using Microsoft.AspNetCore.Mvc;
2
3  namespace MvcLab.Controllers
4  {
5      0 references
6      public class LabController : Controller
7      {
8          // [cite_start]
9          [Route("info")]
10         0 references
11         public IActionResult Info()
12         {
13             // [cite_start]
14             ViewBag.LabNumber = "1";
15             ViewBag.Topic = "Вступ до ASP.NET Core";
16             ViewBag.Purpose = "ознайомитися з основними принципами роботи .NET, навчитися налаштувати середовище розробки та встановлювати необхідні компоненти, набутися навичок створення рішень та проектів різних типів, набутися навичок обробки запитів з використанням middleware.";
17             ViewBag.StudentName = "Денис Грушевицький";
18
19             return View();
20         }
21     }
22 }

```

Рис. 9. Створення Controller з роутом /info який виводить інформацію

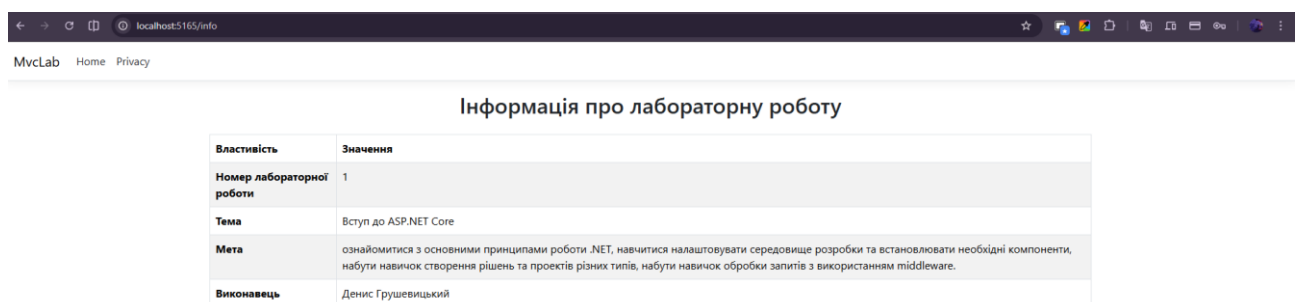


Рис. 10. Вивід сторінки /info

Завдання 3. Робота з middleware

1. Ознайомтеся з [поняттям middleware](#) в ASP.NET
2. Створіть ASP.NET WebAPI проект з назвою MiddlewareSandbox
3. Реалізуйте наступні middleware

```

• PS C:\IPZ-23-4\ASP\lab_1> dotnet new webapi -n MiddlewareSandbox
The template "ASP.NET Core Web API" was created successfully.

Processing post-creation actions...
Restoring C:\IPZ-23-4\ASP\lab_1\MiddlewareSandbox\MiddlewareSandbox.csproj:
Restore succeeded.

• PS C:\IPZ-23-4\ASP\lab_1> cd .\MiddlewareSandbox\

```

Рис. 11. Створення WebAPI проекту

- a. Реалізуйте middleware, яке рахує кількість запитів до сервера і повертає це число у відповіді, наприклад: The amount of processed requests is X.
- b. Створіть middleware, яке аналізує параметри рядка запиту. Якщо в рядку запиту (query string) присутній параметр "custom", то повертати у відповідь "You've hit a custom middleware!", інакше пропускати запит далі. Приклад запиту зображено на рисунку 3.1.
- c. Створіть middleware, яке логуватиме у консоль інформацію про всі запити. В логах відображати метод запиту (GET, POST тощо) та його шлях (/user, /product тощо). Для перевірки POST та інших методів запиту можна скористатися Postman або іншою подібною утилітою.
- d. Реалізуйте middleware, яке перевіряє наявність API-ключа у заголовках запиту. Для цього слід Перевіряти, чи є в запиті заголовок X-API-KEY. Якщо ключ неправильний або відсутній, повертати 403 Forbidden, не передавати запит для виконання далі. Якщо ключ в заголовку запиту співпадає з тим, який заданий на сервері, передавати запит далі. Для додавання відповідного запиту можна скористатися Postman або подібними утилітами.

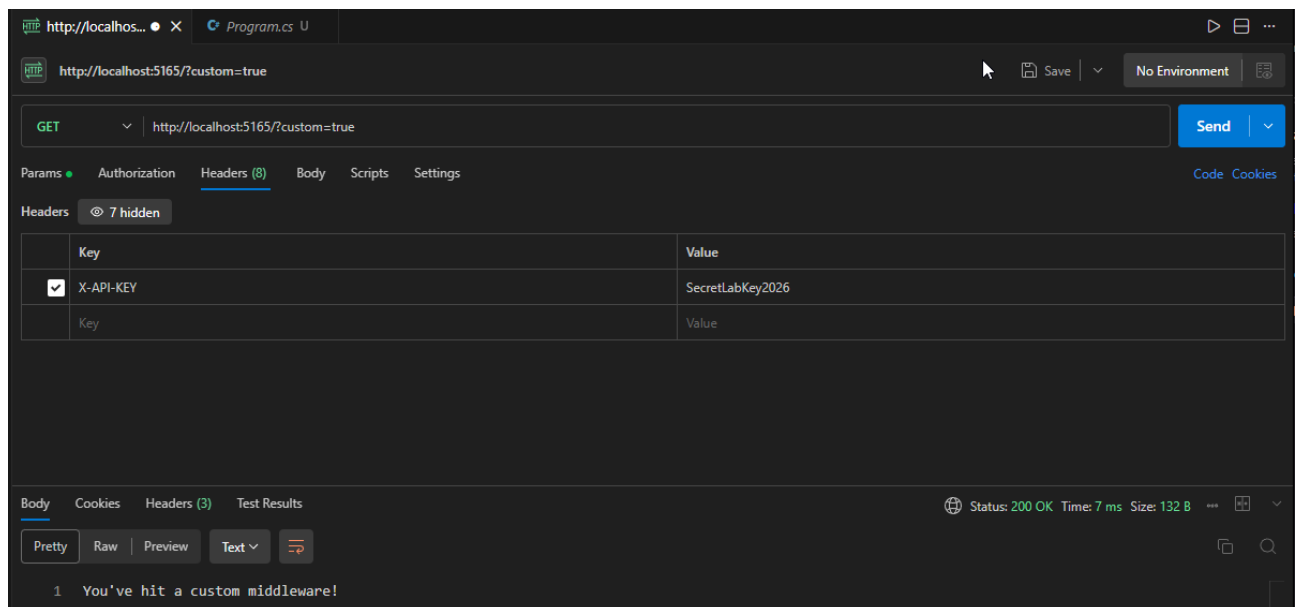


Рис. 12. Реалізований custom параметр

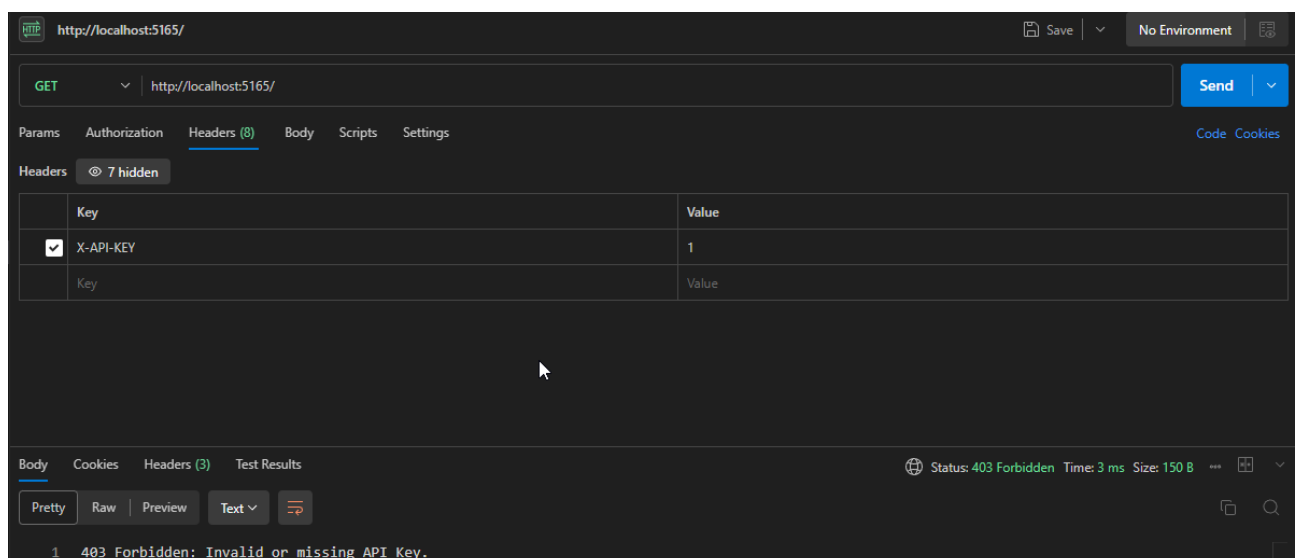


Рис. 13. Потребує ключ для відображення

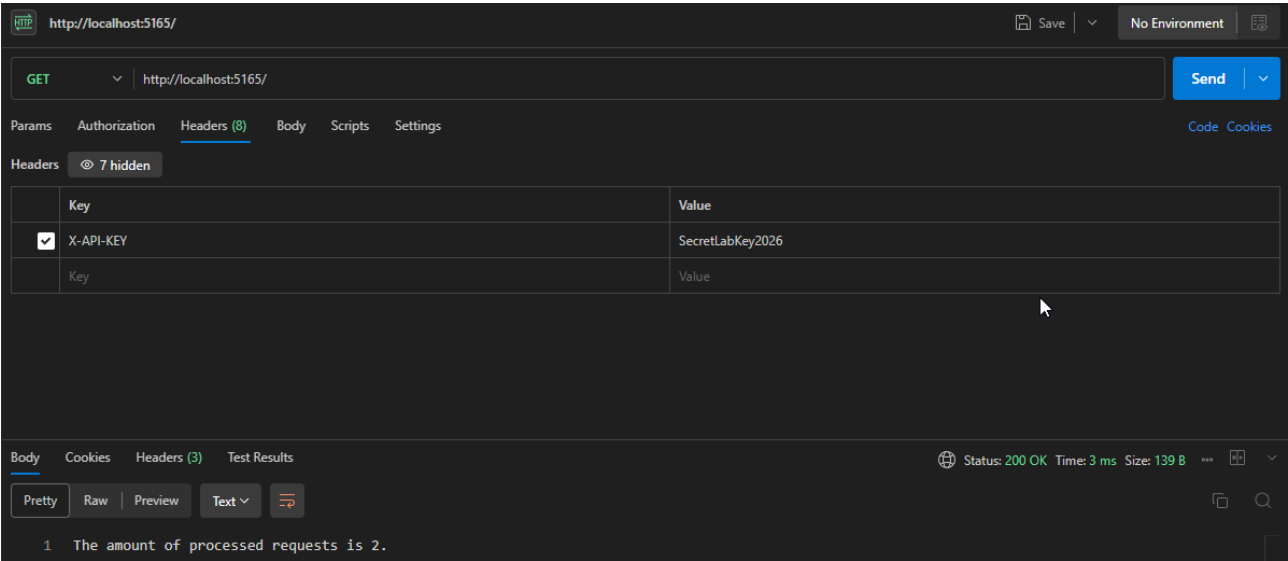


Рис. 14. Відображає кількість запитів коли є правильний API ключ

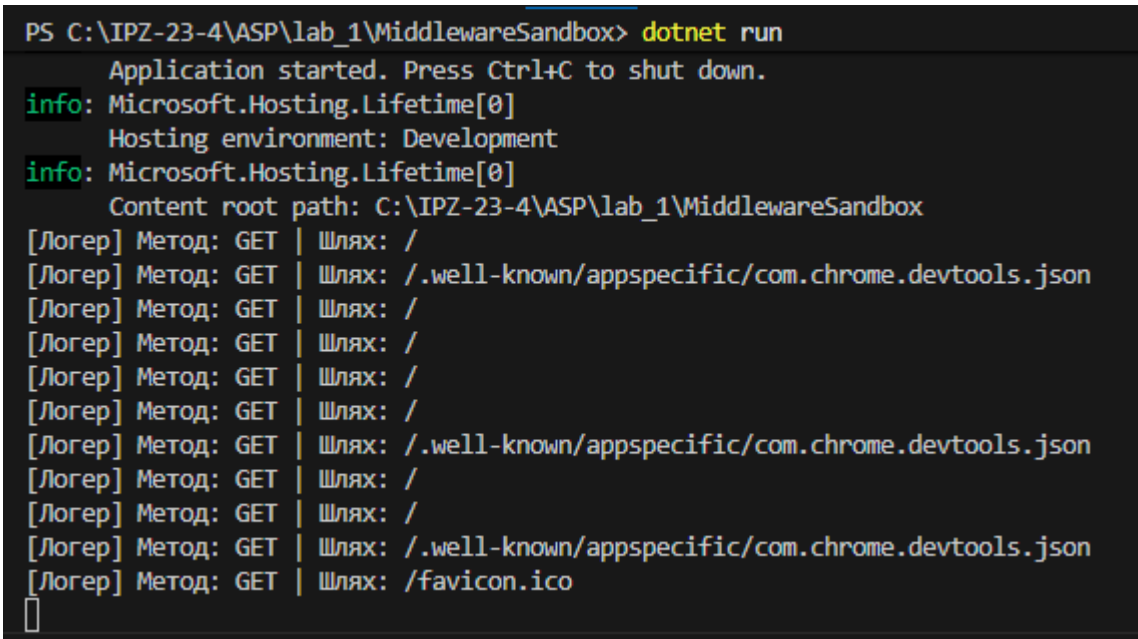


Рис. 15. Логує всі запити і виводить в консоль

Висновок: в ході виконання роботи ознайомився з основними принципами роботи .NET, навчився налаштовувати середовище розробки та встановлювати необхідні компоненти, набув навичок створення рішень та проектів різних типів, набув навичок обробки запитів з використанням middleware.